

DevOpsAgent AI

Autonomous SRE & Infrastructure Self-Healing Engine

Live Cloud Run URL:	https://devops-agent-service-785389711688.us-central1.run.app/
GitHub Repository:	https://github.com/creationsofh/devops-agent
Tech Stack:	Python, FastAPI, Streamlit, Google Cloud Run, Google GenAI SDK

1. EXECUTIVE SUMMARY & OVERVIEW

What is DevOpsAgent AI?

DevOpsAgent AI is an autonomous, production-grade Site Reliability Engineering (SRE) assistant designed to convert raw infrastructure errors, telemetry logs, and stack traces into immediate operational remediation. Unlike standard AI chatbots that only provide advice, DevOpsAgent AI directly executes autonomous code updates, container microservice restarts, and temporary IAM access elevations.

2. TARGETED AUDIENCE & PROBLEM STATEMENT

For Whom is this Built?

- DevOps Engineers & SREs:** Eliminates manual triage for repetitive container crashes and permission blockages.
- Cloud & Full-Stack Developers:** Resolves model deprecation errors and infrastructure bottlenecks without leaving the chat environment.
- On-Call Incident Response Teams:** Accelerates mean-time-to-resolution (MTTR) during active system outages.

Problem Solutions Matrix:

INDUSTRY PAIN POINT	TRADITIONAL IMPACT	DEVOPSAGENT AI SOLUTION
Cascading Model Deprecations	Codebases break silently when LLM API endpoints are deprecated.	Codebase Auto-Healing: Detects version 404s and automatically patches local source files.
Deadlocked Microservices	High MTTR due to manual log inspection and gcloud restarts.	Autonomous Remediation: Parses log stack traces and executes container restarts instantly.

INDUSTRY PAIN POINT	TRADITIONAL IMPACT	DEVOPSAGENT AI SOLUTION
Deployment Access Blocks	Developers stuck waiting for IAM admins during hotfixes.	JIT Access Provisioning: Grants temporary scoped IAM roles upon policy evaluation.

3. SYSTEM ARCHITECTURE & HOW IT WORKS

Decoupled Master Architecture

The system runs as a containerized dual-process microservice on Google Cloud Run. Streamlit manages user interactions on port 8080 while FastAPI processes agent orchestrations on port 8000.

Execution Flow:

- Telemetry Intake:** User inputs incident logs via text or file attachment (.log/.txt up to 10MB).
- FastAPI Orchestration:** The payload is dispatched to the backend, which feeds it to the Google GenAI SDK with structured system prompts.
- Dynamic Function Calling:** If actionable telemetry is identified, Gemini emits structured function calls matching the backend tool map (`update_model_config` `restart_cloud_run_service` `grant_temporary_iam_role`).
- Autonomous Execution & Audit:** The Python backend executes the tool locally or via cloud APIs and returns formatted JSON result cards back to the Streamlit UI.

4. KEY DIFFERENTIATORS

- **Action-Oriented Architecture:** Bypasses pure conversational text output to execute operational remediation directly.
- **Resilient Multi-Model Fallbacks:** Handles API rate limits (HTTP 429) and model availability issues using exponential backoffs and automated candidate target switching.
- **Integrated Visual Audit Cards:** Renders live custom HTML/CSS execution cards detailing function names, arguments, and execution statuses.

5. AUTONOMOUS CAPABILITY SUITE

CAPABILITY	TOOL FUNCTION	TRIGGER SCENARIO
Codebase Self-Repair	update_model_config	Remediates deprecated Gemini API model identifiers across backend/frontend files.
Cloud Run Lifecycle	restart_cloud_run_service	Restarts unresponsive or deadlocked microservice instances across target GCP regions.
IAM Security Elevation	grant_temporary_iam_role	Elevates user permissions temporarily (e.g., roles/run.developer) for urgent hotfixes.

6. COMPREHENSIVE USER MANUAL

A. Testing Infrastructure Self-Healing

Paste the following stack trace into the chat interface:

```
Execution Error: 404 NOT_FOUND. Model models/gemini-2.5-flash is deprecated. Please update code to use models/gemini-3.6-flash.
```

Expected Result: An execution card will render confirming that `update_model_config` patched the local files.

B. Testing Cloud Run Microservice Restart

```
ALERT: order-processing-service in us-central1 is throwing HTTP 500 errors due to process deadlock. Please restart the microservice.
```

Expected Result: Triggers `restart_cloud_run_service` targeting `order-processing-service` in `us-central1`.

C. Testing Advisory Queries

```
What are the best practices for setting up Docker multi-stage builds for FastAPI apps?
```

Expected Result: The agent recognizes no system modification is needed and provides clean architectural guidance.